CCS3202-Assignment 2-YUE CHENGHAO-227154

YUE CHENGHAO

227154

1. Introduction

This assignment demonstrates the implementation of the **Queue Abstract Data Type (ADT)** using:

1. A **Queue interface** that defines all queue operations.
2. An **ArrayQueue class** that implements the interface using a **circular array**.
3. A **test program** that performs a sequence of queue operations and displays:
   - the returned value from each operation, and
   - the remaining contents of the queue after each step.

The goal is to show understanding of ADT design, interface usage, and array-based queue implementation.

---

2. Design Overview

The overall design follows standard object-oriented principles:

- The **Queue interface** defines the public operations:
  `enqueue`, `dequeue`, `front`, `rear`, `isEmpty`, and `size`.
- The **ArrayQueue class** implements these operations using:
  - an integer array to store elements,
  - the `front` index pointing to the first element,
  - the `rear` index pointing to the last element,
  - a `size` variable that tracks the number of elements.

A **circular queue** technique is used so the array space can be reused when elements are removed.

---

3. Test Program

A separate test class performs all the operations listed in the question:

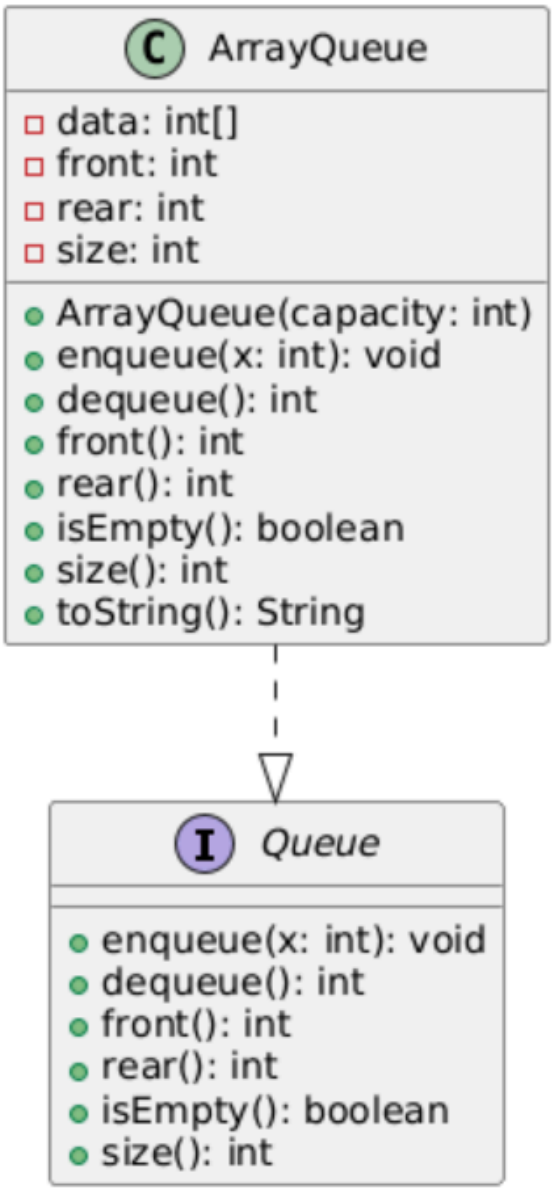
```
enqueue(5)
enqueue(3)
dequeue()
enqueue(7)
dequeue()
front()
dequeue()
dequeue()
isEmpty()
enqueue(9)
enqueue(7)
size()
enqueue(3)
enqueue(5)
dequeue()
rear()
```

For each operation:

- if there is a return value (such as dequeue, front, rear, size), it is printed
- the remaining queue content is also printed

PART 2 — UML CLASS DIAGRAM

Below is the **UML diagram** showing the relationship between the `Queue` interface and the `ArrayQueue` class.



PART 3 — Full Java Code for the Queue ADT (Interface + ArrayQueue)

File 1: `Queue.java` (Interface)

```
public interface Queue {
    void enqueue(int x);   // Insert element into the queue
    int dequeue();         // Remove and return the front element
    int front();           // Return (but do not remove) the front element
    int rear();            // Return (but do not remove) the rear element
    boolean isEmpty();     // Check if the queue is empty
```

```
    int size();            // Return the number of elements in the queue
}
```

File 2: `ArrayQueue.java` (Array-based Implementation of Queue)

```
public class ArrayQueue implements Queue {

    private int[] data;   // Array to store queue elements
    private int front;    // Index of the front element
    private int rear;     // Index of the rear element
    private int size;     // Current number of elements

    // Constructor: create a queue with given capacity
    public ArrayQueue(int capacity) {
        data = new int[capacity];
        front = 0;
        rear = -1;
        size = 0;
    }

    @Override
    public void enqueue(int x) {
        if (size == data.length) {
            System.out.println("Queue is full. Cannot enqueue " + x);
            return;
        }
        // Circular increment of rear
        rear = (rear + 1) % data.length;
        data[rear] = x;
        size++;
    }

    @Override
    public int dequeue() {
        if (isEmpty()) {
            System.out.println("Queue is empty. Cannot dequeue.");
            return Integer.MIN_VALUE;   // Special value indicating failure
        }
        int value = data[front];
        // Circular increment of front
        front = (front + 1) % data.length;
        size--;
        return value;
    }
```

```
    @Override
    public int front() {
        if (isEmpty()) {
            System.out.println("Queue is empty. No front element.");
            return Integer.MIN_VALUE;
        }
        return data[front];
    }

    @Override
    public int rear() {
        if (isEmpty()) {
            System.out.println("Queue is empty. No rear element.");
            return Integer.MIN_VALUE;
        }
        return data[rear];
    }

    @Override
    public boolean isEmpty() {
        return size == 0;
    }

    @Override
    public int size() {
        return size;
    }

    // Helper method to print the queue contents from front to rear
    @Override
    public String toString() {
        if (isEmpty()) {
            return "[]";
        }

        StringBuilder sb = new StringBuilder();
        sb.append("[");
        for (int i = 0; i < size; i++) {
            int index = (front + i) % data.length;
            sb.append(data[index]);
            if (i < size - 1) {
                sb.append(", ");
            }
        }
        sb.append("]");
        return sb.toString();
    }
```

```
}
```

File 3: `TestQueue.java`

```
public class TestQueue {
    public static void main(String[] args) {

        Queue q = new ArrayQueue(10);

        System.out.println("Initial queue: " + q);

        System.out.println("\n1) enqueue(5)");
        q.enqueue(5);
        System.out.println("Queue: " + q);

        System.out.println("\n2) enqueue(3)");
        q.enqueue(3);
        System.out.println("Queue: " + q);

        System.out.println("\n3) dequeue()");
        int r = q.dequeue();
        System.out.println("Return value: " + r);
        System.out.println("Queue: " + q);

        System.out.println("\n4) enqueue(7)");
        q.enqueue(7);
        System.out.println("Queue: " + q);

        System.out.println("\n5) dequeue()");
        r = q.dequeue();
        System.out.println("Return value: " + r);
        System.out.println("Queue: " + q);

        System.out.println("\n6) front()");
        r = q.front();
        System.out.println("Return value: " + r);
        System.out.println("Queue: " + q);

        System.out.println("\n7) dequeue()");
        r = q.dequeue();
        System.out.println("Return value: " + r);
        System.out.println("Queue: " + q);

        System.out.println("\n8) dequeue()");
```

```
        r = q.dequeue();
        System.out.println("Return value: " + r);
        System.out.println("Queue: " + q);

        System.out.println("\n9) isEmpty()");
        boolean empty = q.isEmpty();
        System.out.println("Return value: " + empty);
        System.out.println("Queue: " + q);

        System.out.println("\n10) enqueue(9)");
        q.enqueue(9);
        System.out.println("Queue: " + q);

        System.out.println("\n11) enqueue(7)");
        q.enqueue(7);
        System.out.println("Queue: " + q);

        System.out.println("\n12) size()");
        int s = q.size();
        System.out.println("Return value: " + s);
        System.out.println("Queue: " + q);

        System.out.println("\n13) enqueue(3)");
        q.enqueue(3);
        System.out.println("Queue: " + q);

        System.out.println("\n14) enqueue(5)");
        q.enqueue(5);
        System.out.println("Queue: " + q);

        System.out.println("\n15) dequeue()");
        r = q.dequeue();
        System.out.println("Return value: " + r);
        System.out.println("Queue: " + q);

        System.out.println("\n16) rear()");
        r = q.rear();
        System.out.println("Return value: " + r);
        System.out.println("Queue: " + q);

        System.out.println("\nFinal queue: " + q);
    }
}
```

- **output**

```
1) enqueue(5)
Queue: [5]

2) enqueue(3)
Queue: [5, 3]

3) dequeue()
Return value: 5
Queue: [3]

4) enqueue(7)
Queue: [3, 7]

5) dequeue()
Return value: 3
Queue: [7]

6) front()
Return value: 7
Queue: [7]

7) dequeue()
Return value: 7
Queue: []

8) dequeue()
Queue is empty. Cannot dequeue.
Return value: -2147483648
Queue: []
```

```
9) isEmpty()
Return value: true
Queue: []

10) enqueue(9)
Queue: [9]

11) enqueue(7)
Queue: [9, 7]

12) size()
Return value: 2
Queue: [9, 7]

13) enqueue(3)
Queue: [9, 7, 3]

14) enqueue(5)
Queue: [9, 7, 3, 5]

15) dequeue()
Return value: 9
Queue: [7, 3, 5]

16) rear()
Return value: 5
Queue: [7, 3, 5]

Final queue: [7, 3, 5]
```

Step	Operation	Return Value	Queue After Operation
1	enqu	–	[5]
2	enq	–	[5, 3]
3	dequeue()	**5**	[3]
4	enq	–	[3, 7]
5	dequeue(	**3**	[7]
6	f	**7**	[7]
7	de	**7**	[]
8	dequeue()	**Integer.MIN_VALUE**(empty queue)	[]
9	isEmpty()	**true**	[]
10	enqueue(9)	–	[9]
11	enqueue(7)	–	[9, 7]
12	size()	**2**	[9, 7]
13	enqueue(3)	–	[9, 7, 3]
14	enqueue(5)	–	[9, 7, 3, 5]
15	dequeue()	**9**	[7, 3, 5]
16	rear()	**5**	[7, 3, 5]

- **FINAL STATE**: [7, 3, 5]